Q1.

A.

$$C[n] = \begin{cases} 0 & , \text{ if } n = 0 \\ \min_{c \in coins, c \le n} 1 + C[n-c] & , \text{ if } n > 0 \end{cases}$$

```
min_change_dp(int n) {
dp[n] = [None] * (n+1): memorize solution for cents
coins[] = {1, ...}: The actual coins available
helper(int n) {
    if (n == 0)
        return []
    if dp[n]:
        return dp[n]

    sol = []
    for c in coins
        sol.append([])
        if (n-c) >= 0:
            sub_sol = helper(n-c)
            if len(sub_sol) > 0
                sol[-1].extend(sol-sol)

        sol[-1].append(c)
    dp[n] = sort(sol, lambda x: len(x))[0]
    return dp[n]
}
return helper(n)
}
```

B.

```
def coin_change_dp(int n)
    dp[n] = [None] * (n+1)
    dp[0] = []
    for i = 1 to n:
        sol = []
        for c in coins:
            sol.append([])
            if (i-c) ≥ 0
                sol[-1].concat( dp[i-c], c)
        dp[i] = select sol[i] with min length

    change = [0] * n
    for c in dp[n]:
        change[c] += 1
    return change
```

2.

A.

```
currency-change(int n):
    if (n==0)
        return []
    sol = []
    for b in bills:
        sol.append([])
        if (n-b) ≥ 0:
            sol[-1].concat(currency-change(n-b), b)
    return Non-empty sol[i] with minimum length
```

B:

```
currency-change-dp(int n):
    dp[n] = [None] * (n+1)
    helper(int n)
        if (n==0)
            return []
        if dp[n]: return dp[n]
        sol = []
        for b in bills:
            sol.append([])
            if (n-b) ≥ 0:
                sol[-1].concat(helper(n-b), b)
        dp[n] = Non-empty sol[i] with minimum length
        return dp[n]
    return helper(n)
```

Analysis:

Time complexity = $O(n \cdot k)$

where n is target currency & k is no. of bills available

Space complexity $O(n+1)$

C.

```
Smallest amount: 416
Greedy solution: 7 bills [365, 28, 13, 7, 1, 1, 1]
Optimal DP solution: 5 bills [52, 91, 91, 91, 91]
```

Q3.

B.

$dp[m+1][n+1] = \{0\}$

for $i = 1 \longrightarrow m+1$:

  for $j = 1 \longrightarrow n+1$:

    if $(A[i] == B[j])$

      $dp[i][j] = 1 + dp[i-1][j-1]$

    else

      $\max(dp[i-1][j], dp[i][j-1])$

D.

$$lcs[i][j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ 1 + dp[i-1][j-1], & \text{if } A[i] == B[j] \\ \max(dp[i-1][j], dp[i][j-1]), & \text{otherwise} \end{cases}$$

C. $X = [A, B, C, B, D]$ $y = [B, D, C, A, B]$

B D C A B

		0	1	2	3	4	5
	0	0	0	0	0	0	0
A	1	0	0	0	0	1	1
B	2	0	1	1	1	1	2
C	3	0	1	1	2	2	2
B	4	0	1	1	2	2	3
D	5	0	1	2	2	2	3
			B		C		B

4,5

3,4